*CS319:*

# Internet Programming I

## ***Week3:*** *HTML Styling with CSS*




***Author:*** *Norapat Labchurat*

***Advisor:*** *Chayunpon Vutivorathum*

***Credit:*** *Kwanruthai Kunkitcharoen*


BANGKOK UNIVERSITY
THE CREATIVE UNIVERSITY




# *What is CSS?*

# What Is CSS?

- **CSS** stands for Cascading Style Sheets
- **CSS** describes how **HTML** elements are to be displayed on screen, paper, or in other media
- **CSS** saves a lot of work. It can control the layout of multiple web pages all at once
- External stylesheets are stored in **CSS** files
- **Note:** Child elements inherit its parent element styling **“Cascading (การเรียงซ้อน)”**

https://www.w3schools.com/css/css_intro.asp




# *How to create CSS?*

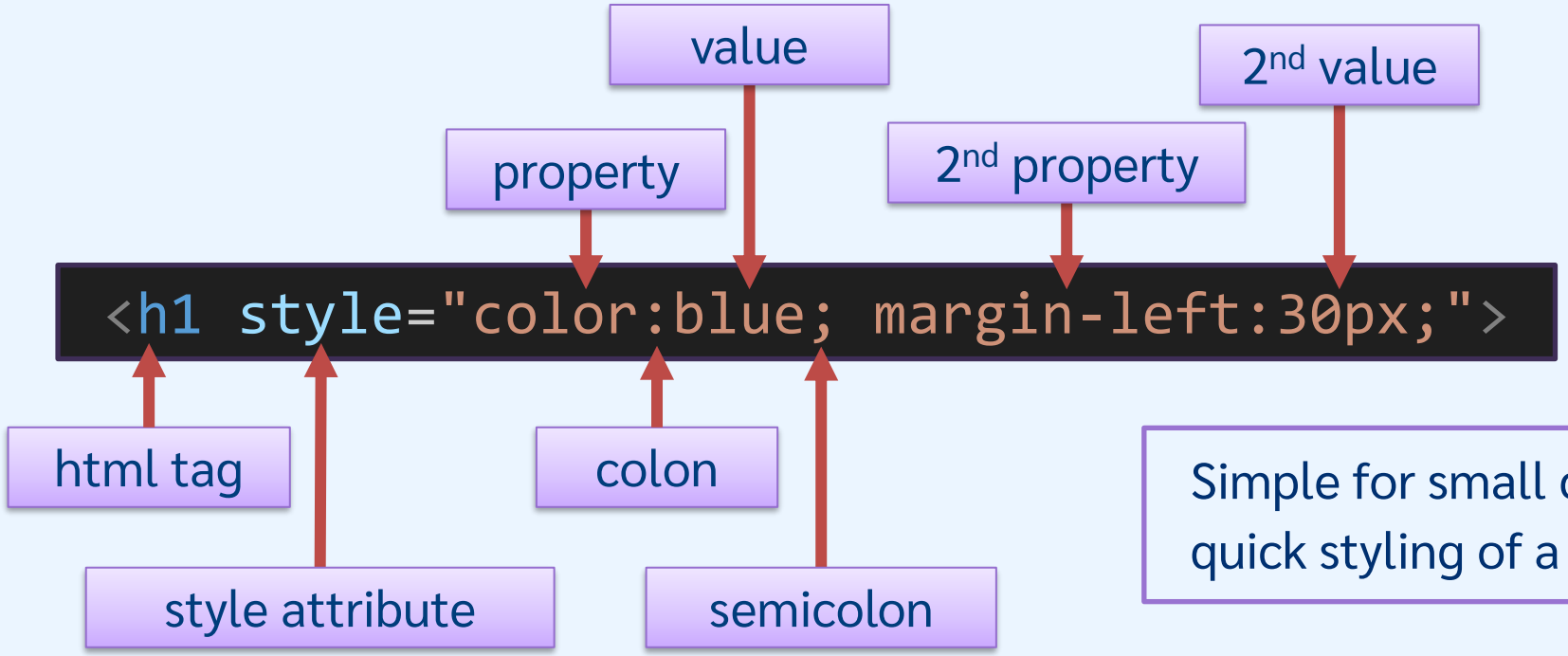
# 1) CSS Inline Style Sheet

Apply style directly within an HTML tag's *style* attribute

```
<h1 style="color:blue; margin-left:30px;">
    This is a heading
</h1>
```

Output




Simple for small changes and quick styling of a single element

# 2) CSS Internal Style Sheet

Defined within the <style> tag in the <head> section of the HTML document

```
<head>
    <style>
        h1 {
            color: blue;
            margin-left: 30px;
        }
    </style>
</head>
<body>
    <h1>This is a heading</h1>
</body>
```

Value

Element name

Property

Html tag

Output



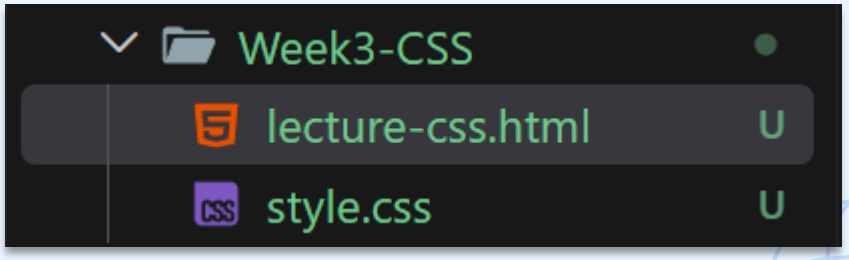

Best for styling to a specific HTML page

# 3) CSS External Style Sheet

Written in a separate .css file and linked to the HTML using a <link> tag

style.css

```css
h1 {
    color: blue;
    margin-left: 30px;
}
```




*Folder Structure*

Lecture-css.html

```html
<head>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>This is a heading</h1>
</body>
```

Relation

CSS file path




*Output*

Ideal for multi-page websites that share a unified design system.

# Applying CSS Priority

**Highest Priority** Inline > Internal > External **Lowest Priority**

Higher priority styles always override lower ones.



# CSS Selectors

CSS Selectors are used to "find" (or select) HTML elements based on *element name*, *name*, *id*, *class*, *attribute*, and more.

*https://www.w3schools.com/css/css_selectors.asp*

# 1) The element Selector

## Using Descendant / Child Combinator

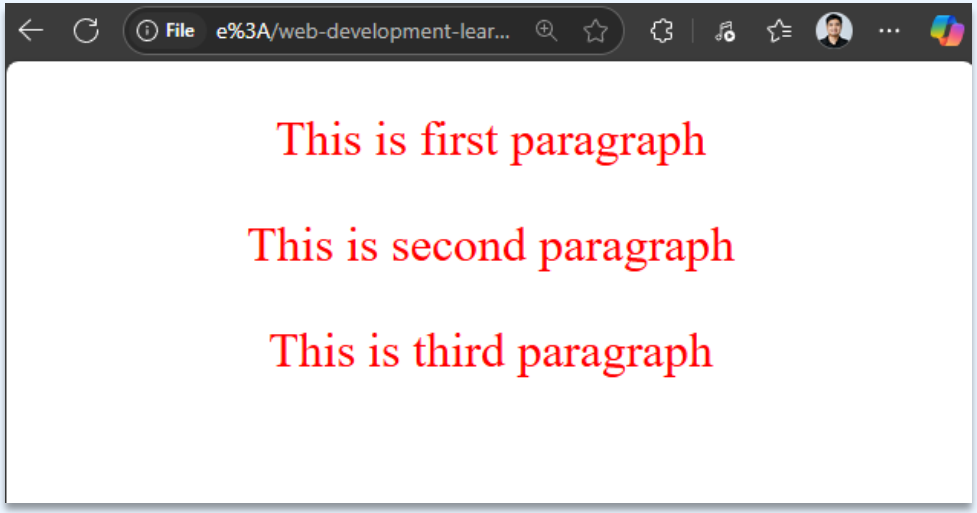
css

```css
p {
  text-align: center;
  color: red;
}
```

You can select all <p> elements on a page like this (in this case, all <p> elements will be center-aligned, with a red text color)

html

```html
<body>
    <p>This is first paragraph</p>
    <p>This is second paragraph</p>
    <p>This is third paragraph</p>
</body>
```




*Output*

# 1) The element Selector (cont.)

Using Descendant / Child Combinator to styling specific inner element

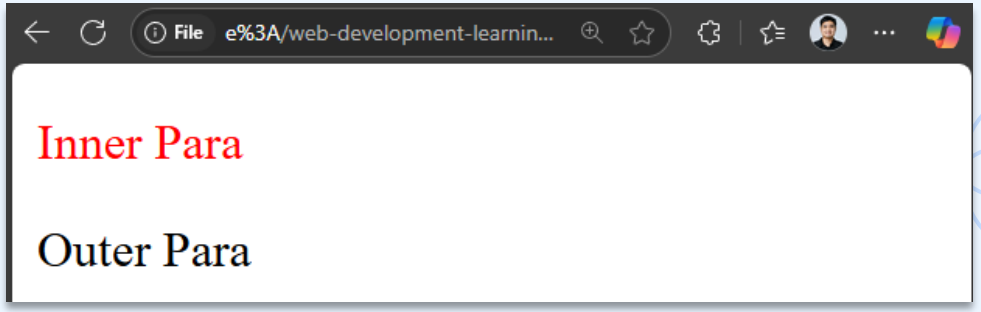
css

```css
article div div p {
    color: red;
}
```

html

```html
<body>
    <article>
        <div>
            <div>
                <p>Inner Para</p>
            </div>
        </div>
    </article>

    <p>Outer Para</p>
</body>
```




*Output*

**However:** Using class is better

# 2) The id Selector

The id selector uses the id attribute of an HTML element to select a specific element.

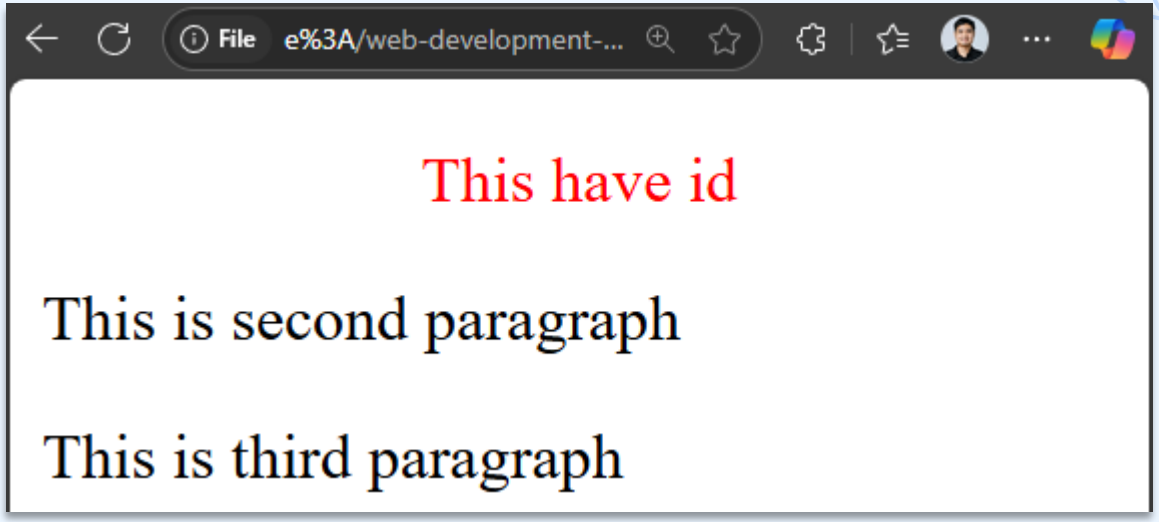
CSS

```css
#para1 {
  text-align: center;
  color: red;
}
```

- The id of an element is unique within a page, so the id selector is used to select one unique element!
- To select an element with a specific id, write a hash (#) character, followed by the id of the element.

html

```html
<body>
    <p id="para1">This have id</p>
    <p>This is second paragraph</p>
    <p>This is third paragraph</p>
</body>
```




*Output*

# 3) The class Selector

The class selector selects HTML elements with a specific class attribute.

css

```css
.center {
  text-align: center;
  color: red;
}
```

To select elements with a specific class, write a period (.) character, followed by the class name.

html

```html
<body>
    <p>This is first</p>
    <p class="center">This is second</p>
    <p class="center">This is third</p>
</body>
```




*Output*

# 3) The class Selector (cont.)

You can also specify that only specific HTML elements should be affected by a class.

css

```css
p.center {
  text-align: center;
  color: red;
}
```

In this example only <p> elements with class="center" will be red and center-aligned:

html

```html
<body>
    <h1 class="center">This is H1</h1>
    <p class="center">This is P</p>
    <h3 class="center">This is H3</h3>
</body>
```




*Output*

# 3) The class Selector (cont.)

HTML elements can also refer to more than one class.

css

```css
.center {
  text-align: center;
  color: red;
}

.large {
  font-size: 32px;
}
```

In this example the <p> element will be styled according to class="center" and to class="large":

html

```html
<body>
    <p class="center large">Has 2 class</p>
    <p class="center">Has 1 class</p>
</body>
```



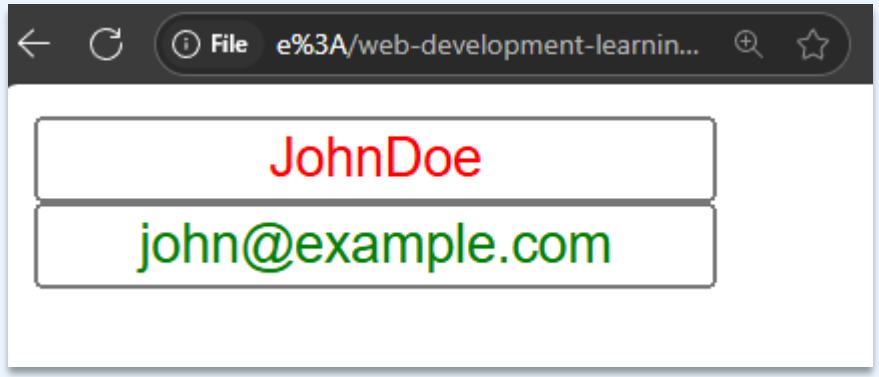

*Output*

# The Attribute Selector

Targeting specific input types using the Attribute Selector [type='...']

css

```css
input[type="text"] {
  text-align: center;
  color: red;
}
input[type="email"] {
  text-align: center;
  color: green;
}
```




*Output*

html

```html
<body>
    <input type="text" value="JohnDoe"><br>
    <input type="email" value="john@example.com">
</body>
```

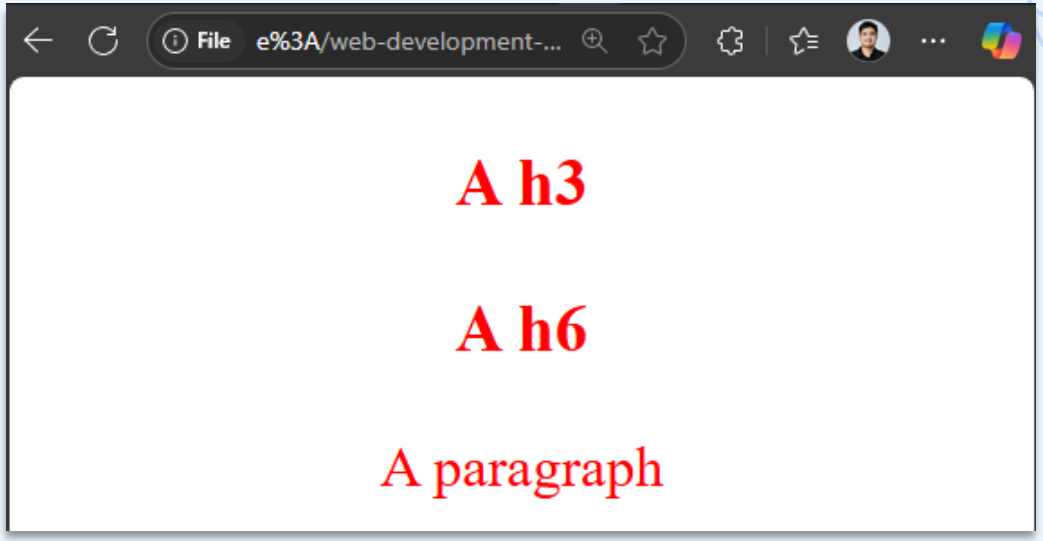
# CSS Universal Selector

The universal selector (*) selects all HTML elements on the page.

```css
* {
  text-align: center;
  color: red;
}
```

```html
<body>
    <H3>A h3</H3>
    <H3>A h6</H3>
    <p>A paragraph</p>
</body>
```

This CSS rule will affect every HTML element on the page:




*Output*

# CSS Grouping Selector

The grouping selector selects all the HTML elements with the same style definitions.

Ungroup

```
h1 {
  text-align: center;
  color: red;
}

h2 {
  text-align: center;
  color: red;
}

p {
  text-align: center;
  color: red;
}
```

- It will be better to group the selectors, to minimize the code.
- To group selectors, separate each selector with a comma.

Group

```
h1, h2, p {
  text-align: center;
  color: red;
}
```

# CSS Pseudo-classes

A pseudo-class is used to define a special state of an element (e.g., when hovered, focused, or targeting specific children).

```css
button:hover {
  background-color: #3182ce;
  color: #ffffff;
}
input:focus {
  border-color: #3182ce;
  outline: none;
}
a:visited {
  color: #805ad5;
}
li:first-child {
  font-weight: bold;
}
li:nth-child(even) {
  background-color: #f7fafc;
}
```

- :hover: Applies styles when the user moves the mouse pointer over the element.
- :focus: Applies styles when an element receives focus (e.g., clicking inside an <input>).
- :active: Applies styles during the exact moment an element is being clicked/pressed.
- :visited: Styles links (<a>) that the user has already visited in their browser.
- :first-child / :last-child: Targets the very first or last sibling element inside a parent container.
- :nth-child(n): Selects elements based on a formula or keyword (e.g., 2, odd, even, 3n).

# CSS Transition

Using transition to smoothly transitions both when entering and leaving the state.

```
.btn {
  /* Specific Property Transitions */
  transition: background-color 0.25s ease, transform 0.25s ease;
}
```

- Always apply transition to the base class (.btn), not the pseudo-class (like :hover), so the animation is smooth both on mouse-enter and mouse-leave.
- 0.25s sets the animation duration (a quarter of a second) for a quick, responsive feel.
- ease defines the timing function (slow start, fast middle, slow finish) to make the motion look natural.
- Listing specific properties (background-color, transform) improves performance compared to using transition: all



# CSS Properties

A CSS property is a parameter used in a CSS declaration that lets you style certain aspects of selected elements.

*https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties*

# Typography & Text Properties

```
p {
    font-family: Arial, Helvetica, sans-serif;
    font-size: 16px;
    font-weight: bold;
    font-style: italic;
    color: #2b6cb0;
    text-align: center;
    line-height: 1.6;
    letter-spacing: 1px;
    text-transform: uppercase;
    text-decoration: underline;
    text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.2);
}
```

- font-family: Sets the typeface list or single font name.
- font-size: Controls the text magnitude (e.g., px, em, rem).
- font-weight: Adjusts text thickness (e.g., bold, 400).
- font-style: Controls italicization (e.g., normal, italic).
- color: Determines the text foreground color.
- text-align: Handles horizontal text alignment (e.g., left, center, right, justify).
- line-height: Modifies vertical text line spacing.
- letter-spacing: Horizontal tracking between characters.
- text-transform: Controls character capitalization (e.g., uppercase, lowercase, capitalize).
- text-decoration: Adds visual decorative lines to text (e.g., underline, line-through, none).
- text-shadow: Adds drop shadows behind text (offset-x, offset-y, blur-radius, color).

# 🖼️ Backgrounds & Surfaces

```
.hero-banner {
  background-color: #2d3748;
  background-image: url("banner.jpg");
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;
}
```

- background-color: Sets a fallback solid color backdrop.
- background-image: Applies an image or gradient (e.g., url(...), linear-gradient(...)).
- background-size: Controls image scaling behavior (cover, contain, 100%).
- background-position: Sets the anchor coordinates for the graphic (center, top left).
- background-repeat: Controls whether the background tiles across empty space (no-repeat, repeat-x).

# 🎨 Visual Effects & Shapes

```
.glass-card {
  border-radius: 12px;
  box-shadow: 0 10px 20px rgba(0, 0, 0, 0.15);
  opacity: 0.95;
  filter: blur(0px) brightness(1.1);
}
```

- border-radius: Curves sharp container corners into rounded edges.
- box-shadow: Adds multi-layer drop shadows or inset depth (x-offset, y-offset, blur, color).
- opacity: Sets overall element and content transparency from 0.0 to 1.0.
- filter: Applies visual post-processing effects like blur(), grayscale(), or brightness().

# 📦 Box Model & Sizing

```css
.card {
  box-sizing: border-box;
  /* Dimensions & Constraints */
  width: 100%;
  max-width: 400px;
  min-height: 200px;

  /* Spacing */
  margin: 20px auto;
  padding: 16px;

  /* Border & Surface */
  border: 2px solid #3182ce;
  border-radius: 8px;
}
```

- box-sizing: Defines how total width and height are calculated (e.g., border-box includes padding and border within the set width/height).
- width / height: Sets the horizontal and vertical dimensions.
- max-width / max-height: Establishes maximum dimensions to prevent elements from scaling too large.
- min-width / min-height: Establishes minimum dimensions to prevent elements from shrinking too small.
- margin: Shorthand for controlling the outer space around an element.
- padding: Shorthand for controlling the inner space between content and its border.
- border: Shorthand for setting border width, line style, and color.
- border-radius: Rounds the outer corners of border box.

# ⚓ Positioning & Flow Control

```css
/* Positioned Floating Badge */
.badge {
  position: absolute;
  top: 12px;
  right: 12px;
  z-index: 10;
}

/* Flow Control / Text Wrapping */
.thumbnail {
  float: left;
  margin-right: 12px;
}

.clear-divider {
  clear: both;
}
```

- position: Defines the positioning method (static, relative, absolute, fixed, sticky).
- top / right / bottom / left: Sets exact offset distances from the reference element edges.
- z-index: Controls the stacking order along the z-axis for overlapping positioned elements.
- overflow: Manages content that exceeds box boundaries (hidden, scroll, auto).
- float / clear: Controls media text wrapping and prevents elements from wrapping around floated items.

# Modern Multi-Element Layouts (Flexbox & Grid)

```
/* Flexbox Layout Component */
.nav-bar {
  display: flex;
  flex-direction: row;
  justify-content: space-between;
  align-items: center;
  gap: 16px;
}

/* CSS Grid Layout Component */
.card-grid {
  display: grid;
  grid-template-columns: repeat(3,
1fr);
  grid-template-rows: auto;
  gap: 20px;
}
```

- display: Switches the element's layout mode (e.g., flex, grid, block, inline).
- flex-direction: Defines the direction of the main axis in Flexbox (row, column).
- justify-content: Aligns items along the main axis (flex-start, center, space-between).
- align-items: Aligns items along the cross axis (stretch, center, flex-end).
- gap: Adds consistent gutter spacing between flex or grid items without using margins.
- grid-template-columns / grid-template-rows: Defines track columns and rows in CSS Grid (e.g., repeat(3, 1fr)).

# More Styling

## 🎬 Animations & Transitions

These properties bring interactions and motion graphics to life.

- transition: Shorthand for smoothly animating property updates over time.
- transform: Modifies elements in 2D or 3D space (rotate(), scale(), translate()).
- animation: Shorthand configuration to link @keyframes sequences to elements.

## 📜 List Styling

These properties specifically alter HTML lists (<ul> and <ol>).

- list-style-type: Sets the bullet or numbering design (e.g., disc, square, decimal, none).
- list-style-position: Places markers inside or outside the text flow layout.
- list-style-image: Replaces standard markers with a custom image asset.

# CSS Frameworks

## Utility Frameworks

tailwindcss

- Utility-first, allows for full customization
- High level of customization and flexibility
- Steeper, requires understanding of CSS
- Smaller, as unused styles can be purged
- Full control over design and layout
- Flexbox-based grid system
- No pre-designed components

## Component Frameworks

BOOTSTRAP

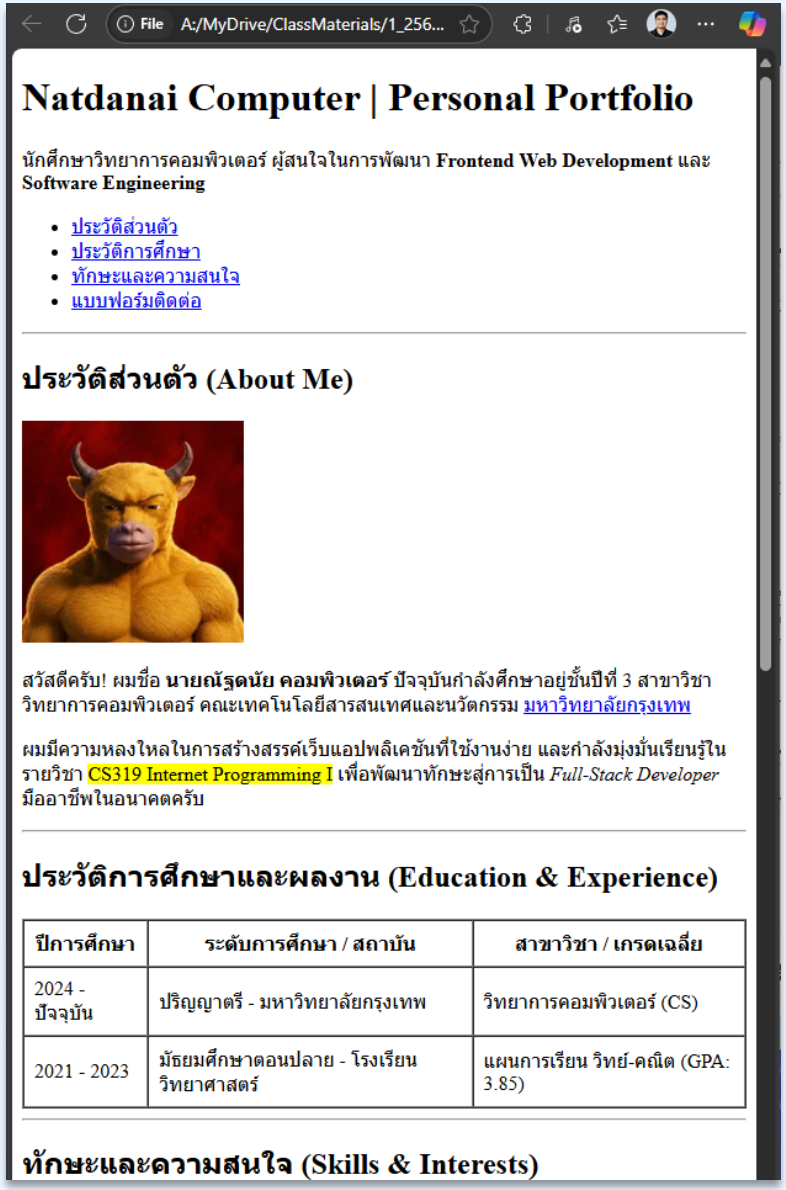
- Predefined components and styles
- Limited flexibility, relies on pre-designed components
- Easier, built-in classes and components
- Larger, as it includes all components by default
- Limited control without overriding default styles
- 12-column grid system with predefined widths
- Offers a variety of pre-designed components such as buttons, modals, forms



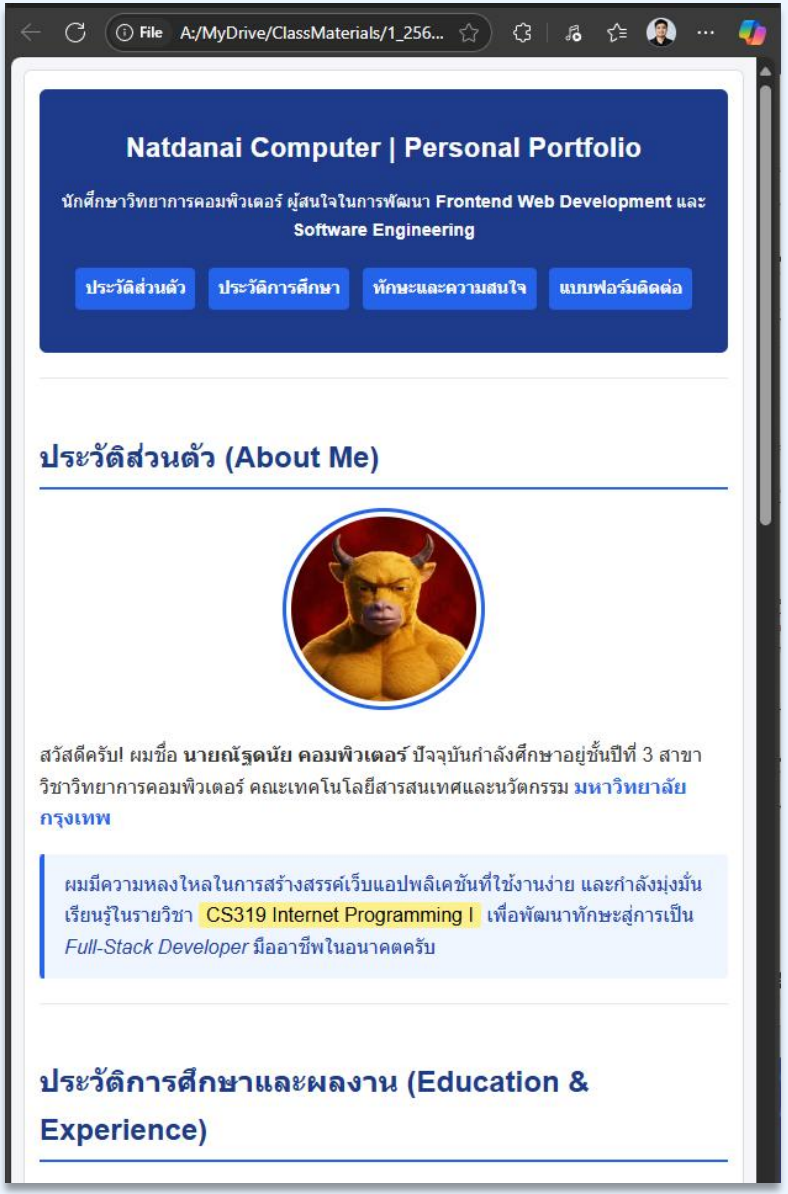

# Deploy You Web on Netlify

a cloud platform that helps developers build, deploy, and scale modern web applications and static websites directly from a Git repository

# Class Activity – Styling Your Portfolio




Apply CSS



# Themes and templates

Free Bootstrap Themes, Templates, Snippets, and Guides - Start Bootstrap

Free & Premium Responsive Bootstrap Templates @ ThemeWagon

64+ HTML5 Website Templates

W3.CSS Templates

# Research further

http://www.w3schools.com/css/default.asp

https://10015.io/

https://lordicon.com/

https://www.30secondsofcode.org/

https://www.w3schools.com/css/css_website_layout.asp

## Check This Week Assignment on

# *MS Team*

***Thank you!***